

Práctica de Programación - Clasificación de textos en Esperanto

Minería de textos - 4 INF-A - 2025/2026

Javier Paris

2025-09-22

Table of contents

1	Objetivos	2
2	Herramientas	2
3	Fase 1 — Repositorio	2
3.1	Gestión del repositorio	2
3.2	Construcción del dataset	3
3.3	Librería de NLP en Esperanto	3
4	Fase 2 — Cuaderno	3
4.1	Entrenamiento del modelo de clasificación	3
4.2	Clasificación de páginas web	3
5	Requisitos para la entrega	4
6	Entrega	4
7	Evaluación	4
8	Notas	4
9	Extra (hasta +2 puntos)	5
9.1	Gestión avanzada del repositorio	5
9.2	Buen rendimiento del modelo	5
9.3	Independencia del dataset	5
9.4	Independencia de la página web	5

Eres un ingeniero lingüístico y te enfrentas a un reto: el Esperanto, la lengua construida más hablada del mundo, apenas cuenta con herramientas de Procesamiento de Lenguaje Natural. Tu misión es construir, de principio a fin, un sistema capaz de **clasificar textos en Esperanto por temática**: desde la recopilación de datos, pasando por tu propia librería de NLP, hasta la

clasificación de páginas web reales. Para ello usarás técnicas de web scraping, consumo de APIs, NLP y aprendizaje automático con Python.

Deberás elegir **como mínimo 3 categorías temáticas** sobre las que clasificar (por ejemplo: TODO: añadir ejemplos (países/regiones/ciudades ; mamíferos/aves/reptiles)) Las categorías deben ser **semánticamente cercanas** entre sí, de modo que la clasificación no resulte trivial. Las categorías deben acordarse previamente con el profesor.

Número de integrantes: **equipos de máximo 3 personas** (preferiblemente 3).

1 Objetivos

- Poner en práctica el temario de la asignatura: minería de textos, PLN y minería web.
- Formalizar conocimiento lingüístico de forma que admita un tratamiento algorítmico (CE25).
- Aplicar técnicas de aprendizaje computacional para la extracción automática de información a partir de grandes volúmenes de datos (CE26).
- Adquirir experiencia con flujos de trabajo profesionales: repositorios, librerías, documentación y pruebas.

2 Herramientas

La práctica se divide en dos bloques con entornos distintos:

- El **repositorio** y la **librería de NLP** se desarrollarán en **Python** y se alojarán en **GitHub**.
- El **entrenamiento del modelo** y la **clasificación web** se desarrollarán en un único cuaderno Jupyter de Python (.ipynb), aconsejable sobre [Google Colab](#).
- Se permite el uso de cualquier librería vista en clase (**requests**, **beautifulsoup4**, **pandas**, **scikit-learn**, **matplotlib**...).
- Se permite el uso de herramientas de IA (Gemini u otras) como apoyo, nunca como sustituto del alumno. El alumno debe ser consciente, crítico y ser capaz de defender todas las decisiones y resultados del proyecto.

3 Fase 1 — Repositorio

Las subfases de construcción del dataset y de la librería pueden desarrollarse en paralelo.

3.1 Gestión del repositorio

Se creará y gestionará un repositorio de GitHub que albergará el *script* de recogida de datos, el dataset y la librería. Debe reflejar un uso ordenado del control de versiones: estructura clara, *commits* significativos y contribución equilibrada de todos los integrantes.

Resultado: un repositorio de GitHub organizado y documentado.

3.2 Construcción del dataset

A partir del código proporcionado para la API de Wikipedia, se construirá un *script* que recupere texto de las categorías elegidas, lo divida en párrafos y genere el dataset etiquetado conforme al *Contrato de datos*.

Heredar la etiqueta del artículo introduce **ruido**: hay párrafos genéricos que no distinguen la categoría (p. ej. “*Ĝi estis fondita en 1950*”). Reconocer, comentar y, en la medida de lo posible, medir ese ruido forma parte del trabajo.

Resultado: un dataset etiquetado en Esperanto (`text;class`) y el *script* reproducible que lo genera, alojados en GitHub.

3.3 Librería de NLP en Esperanto

Se desarrollará una librería de Python de NLP para Esperanto con una **API similar a la de spaCy**, aprovechando la regularidad gramatical de la lengua. La librería debe ser **basada en reglas** e incluir, como mínimo:

- Tokenizador.
- Lematizador.
- Etiquetador morfológico / gramatical (POS).
- Gestión de *stop-words*.
- Objetos tipo `Doc` / `Token` que estructuren el resultado.

Resultado: una librería de Python instalable, con documentación y pruebas.

4 Fase 2 — Cuaderno

4.1 Entrenamiento del modelo de clasificación

Usando la librería de la Fase 1 para procesar el texto, se construirá una representación tabular del corpus con técnicas vistas en clase y se entrenará un modelo de clasificación. El algoritmo es de libre elección, siempre que su entrenamiento y uso no requieran grandes recursos computacionales.

Debe seguirse la metodología estándar de aprendizaje automático: preprocesamiento, partición *train/test* o validación cruzada, y evaluación con métricas adecuadas (*accuracy*, F1 macro y matriz de confusión).

Resultado: un modelo entrenado, junto con su evaluación de rendimiento.

4.2 Clasificación de páginas web

Mediante web scraping, se extraerá el texto de una página web y se determinará, con el modelo entrenado, la probabilidad de pertenecer a cada categoría. Sirve como prueba de funcionamiento del sistema completo sobre datos reales.

Resultado: el mismo cuaderno, ampliado con la obtención y clasificación de texto web.

5 Requisitos para la entrega

- El cuaderno debe estar desarrollado en un fichero `.ipynb` y ser ejecutable en Google Colab.
- El cuaderno debe ser **autocontenido**: no debe depender de ficheros locales ni de interacción del usuario. Los datos y la librería se alojarán en un repositorio de Github público y se descargarán automáticamente desde el cuaderno.
- El código debe ejecutarse de forma autónoma y ser capaz de sobrevivir a un fallo en alguna de las webs utilizadas.
- Para el desarrollo se deberán usar web scraping, APIs y técnicas de NLP vistas en clase.
- El código será probado por el profesor en Google Colab.

6 Entrega

La entrega tiene dos hitos:

- **Punto de control (mitad de la práctica)**: enlace al repositorio de GitHub con el *script*, el dataset y la librería (Fase 1). Fecha: *[por determinar]*.
- **Entrega final**: cuaderno `.ipynb` con el código y los resultados (Fase 2). Fecha: *[por determinar]*.

7 Evaluación

La práctica se defenderá **por todo el equipo** ante el profesor. El equipo deberá explicar el desarrollo, las decisiones de diseño y los resultados, responder a preguntas sobre el código y ser capaz de realizar pequeñas modificaciones que se soliciten en el momento. La calificación del equipo es **compartida**.

La práctica se puntúa del siguiente modo (sobre 10):

- Gestión del repositorio — **1 punto**
- Construcción del dataset — **1,5 puntos**
- Librería de NLP — **3,5 puntos**
- Modelo de clasificación — **2,5 puntos**
- Clasificación de páginas web — **1,5 puntos**

8 Notas

- Todos los integrantes deben asistir a la defensa. Quien no asista obtendrá un 0 en la práctica.
- Los resultados deben ser reproducibles: fijar semillas aleatorias y versiones de las APIs y librerías utilizadas.
- Si el tiempo y las circunstancias lo permiten, y con el consentimiento del equipo, la práctica podrá defenderse en clase frente al resto de compañeros, teniéndose en cuenta positivamente para la nota de participación.
- La nota máxima de la práctica es de 10 puntos.

9 Extra (hasta +2 puntos)

Existen 4 apartados adicionales que sumarán hasta **0.5 puntos** cada uno en caso de resolverse correctamente.

9.1 Gestión avanzada del repositorio

El repositorio debe contar con una página *Read the Docs* con la documentación de la librería y un flujo de trabajo de *GitHub Actions* que ejecute automáticamente las pruebas unitarias de la librería cada vez que se haga un *push* al repositorio.

9.2 Buen rendimiento del modelo

El modelo de clasificación debe obtener resultados sólidos, bien justificados. Realizar un análisis exhaustivo del modelo entrenado, con métricas, visualizaciones, ejemplos de predicciones correctas e incorrectas, y conclusiones.

9.3 Independencia del dataset

El dataset del equipo debe cumplir el *Contrato de datos*: **CSV** con codificación **UTF-8**, separador **;**, dos columnas con cabecera **text;class**, granularidad de párrafo y categorías semánticamente cercanas. El cuaderno debe ser capaz de reentrenar y evaluar el modelo de clasificación sobre el dataset de cualquier otro equipo que cumpla dicho contrato.

9.4 Independencia de la página web

El cuaderno debe ser capaz de clasificar correctamente el texto de cualquier página web proporcionada, sin depender de la estructura de las webs utilizadas durante el desarrollo.